Arnoldi iteration

In numerical linear algebra, the **Arnoldi iteration** is an eigenvalue algorithm and an important example of an iterative method. Arnoldi finds an approximation to the eigenvalues and eigenvectors of general (possibly non-Hermitian) matrices by constructing an orthonormal basis of the Krylov subspace, which makes it particularly useful when dealing with large sparse matrices.

The Arnoldi method belongs to a class of linear algebra algorithms that give a partial result after a small number of iterations, in contrast to so-called *direct methods* which must complete to give any useful results (see for example, Householder transformation). The partial result in this case being the first few vectors of the basis the algorithm is building.

When applied to Hermitian matrices it reduces to the Lanczos algorithm. The Arnoldi iteration was invented by W. E. Arnoldi in 1951.[1]

Krylov subspaces and the power iteration

An intuitive method for finding the largest (in absolute value) eigenvalue of a given $m \times m$ matrix $\boldsymbol{A}$ is the power iteration: starting with an arbitrary initial vector b, calculate Ab, A^2b, A^3b, ... normalizing the result after every application of the matrix A.

This sequence converges to the eigenvector corresponding to the eigenvalue with the largest absolute value, $\boldsymbol{\lambda_1}$. However, much potentially useful computation is wasted by using only the final result, $\boldsymbol{A^{n-1}b}$. This suggests that instead, we form the so-called *Krylov matrix*:

$$\boldsymbol{K_n = \begin{bmatrix} b & Ab & A^2b & \cdots & A^{n-1}b \end{bmatrix}.}$$

The columns of this matrix are not in general orthogonal, but we can extract an orthogonal basis, via a method such as Gram–Schmidt orthogonalization. The resulting set of vectors is thus an orthogonal basis of the *Krylov subspace*, $\boldsymbol{\mathcal{K}_n}$. We may expect the vectors of this basis to span good approximations of the eigenvectors corresponding to the $\boldsymbol{n}$ largest eigenvalues, for the same reason that $\boldsymbol{A^{n-1}b}$ approximates the dominant eigenvector.

The Arnoldi iteration

The Arnoldi iteration uses the modified Gram–Schmidt process to produce a sequence of orthonormal vectors, q_1, q_2, q_3, ..., called the *Arnoldi vectors*, such that for every n, the vectors q_1, ..., q_n span the Krylov subspace $\boldsymbol{\mathcal{K}_n}$. Explicitly, the algorithm is as follows:

```
Start with an arbitrary vector q_1 with norm 1.
Repeat for k = 2, 3, ...
  q_k := A q_{k-1}
  for j from 1 to k - 1
    h_{j,k-1} := q_j^* q_k
    q_k := q_k - h_{j,k-1} q_j
  h_{k,k-1} := ‖q_k‖
  q_k := q_k / h_{k,k-1}
```

The j-loop projects out the component of $\boldsymbol{q_k}$ in the directions of $\boldsymbol{q_1, \ldots, q_{k-1}}$. This ensures the orthogonality of all the generated vectors.

The algorithm breaks down when q_k is the zero vector. This happens when the minimal polynomial of A is of degree k. In most applications of the Arnoldi iteration, including the eigenvalue algorithm below and GMRES, the algorithm has converged at this point.

Every step of the k-loop takes one matrix-vector product and approximately $4mk$ floating point operations.

In the programming language Python with support of the NumPy library:

```python
import numpy as np

def arnoldi_iteration(A, b, n: int):
    """Compute a basis of the (n + 1)-Krylov subspace of the matrix A.

    This is the space spanned by the vectors {b, Ab, ..., A^n b}.

    Parameters
    ----------
    A : array_like
        An m × m array.
    b : array_like
        Initial vector (length m).
    n : int
        One less than the dimension of the Krylov subspace, or
equivalently the *degree* of the Krylov space. Must be >= 1.

    Returns
    -------
    Q : numpy.array
        An m x (n + 1) array, where the columns are an orthonormal
basis of the Krylov subspace.
    h : numpy.array
        An (n + 1) x n array. A on basis Q. It is upper Hessenberg.
    """
```

```
    eps = 1e-12
    h = np.zeros((n + 1, n))
    Q = np.zeros((A.shape[0], n + 1))
    # Normalize the input vector
    Q[:, 0] = b / np.linalg.norm(b, 2)  # Use it as the first Krylov
vector
    for k in range(1, n + 1):
        v = np.dot(A, Q[:, k - 1])  # Generate a new candidate vector
        for j in range(k):  # Subtract the projections on previous
vectors
            h[j, k - 1] = np.dot(Q[:, j].conj(), v)
            v = v - h[j, k - 1] * Q[:, j]
        h[k, k - 1] = np.linalg.norm(v, 2)
        if h[k, k - 1] > eps:  # Add the produced vector to the list,
unless
            Q[:, k] = v / h[k, k - 1]
        else:  # If that happens, stop iterating.
            return Q, h
    return Q, h
```

Properties of the Arnoldi iteration

Let Q_n denote the m-by-n matrix formed by the first n Arnoldi vectors q_1, q_2, ..., q_n, and let H_n be the (upper Hessenberg) matrix formed by the numbers $h_{j,k}$ computed by the algorithm:

$$\boldsymbol{H_n = Q_n^* A Q_n}.$$

The orthogonalization method has to be specifically chosen such that the lower Arnoldi/Krylov components are removed from higher Krylov vectors. As $\boldsymbol{Aq_i}$ can be expressed in terms of q_1, ..., q_{i+1} by construction, they are orthogonal to q_{i+2}, ..., q_n.

We then have

$$\boldsymbol{H_n} = \begin{bmatrix} h_{1,1} & h_{1,2} & h_{1,3} & \cdots & h_{1,n} \\ h_{2,1} & h_{2,2} & h_{2,3} & \cdots & h_{2,n} \\ 0 & h_{3,2} & h_{3,3} & \cdots & h_{3,n} \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & h_{n,n-1} & h_{n,n} \end{bmatrix}.$$

The matrix H_n can be viewed as A in the subspace $\boldsymbol{\mathcal{K}_n}$ with the Arnoldi vectors as an orthogonal basis; A is orthogonally projected onto $\boldsymbol{\mathcal{K}_n}$. The matrix H_n can be characterized by the following optimality condition. The characteristic polynomial of H_n minimizes $\|p(A)q_1\|_2$ among all monic polynomials of degree n. This optimality problem has a unique solution if and only if the Arnoldi iteration does not break down.

The relation between the Q matrices in subsequent iterations is given by

$$AQ_n = Q_{n+1}\tilde{H}_n$$

where

$$\tilde{H}_n = \begin{bmatrix} h_{1,1} & h_{1,2} & h_{1,3} & \cdots & h_{1,n} \\ h_{2,1} & h_{2,2} & h_{2,3} & \cdots & h_{2,n} \\ 0 & h_{3,2} & h_{3,3} & \cdots & h_{3,n} \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ \vdots & & 0 & h_{n,n-1} & h_{n,n} \\ 0 & \cdots & \cdots & 0 & h_{n+1,n} \end{bmatrix}$$

is an $(n+1)$-by-n matrix formed by adding an extra row to H_n.

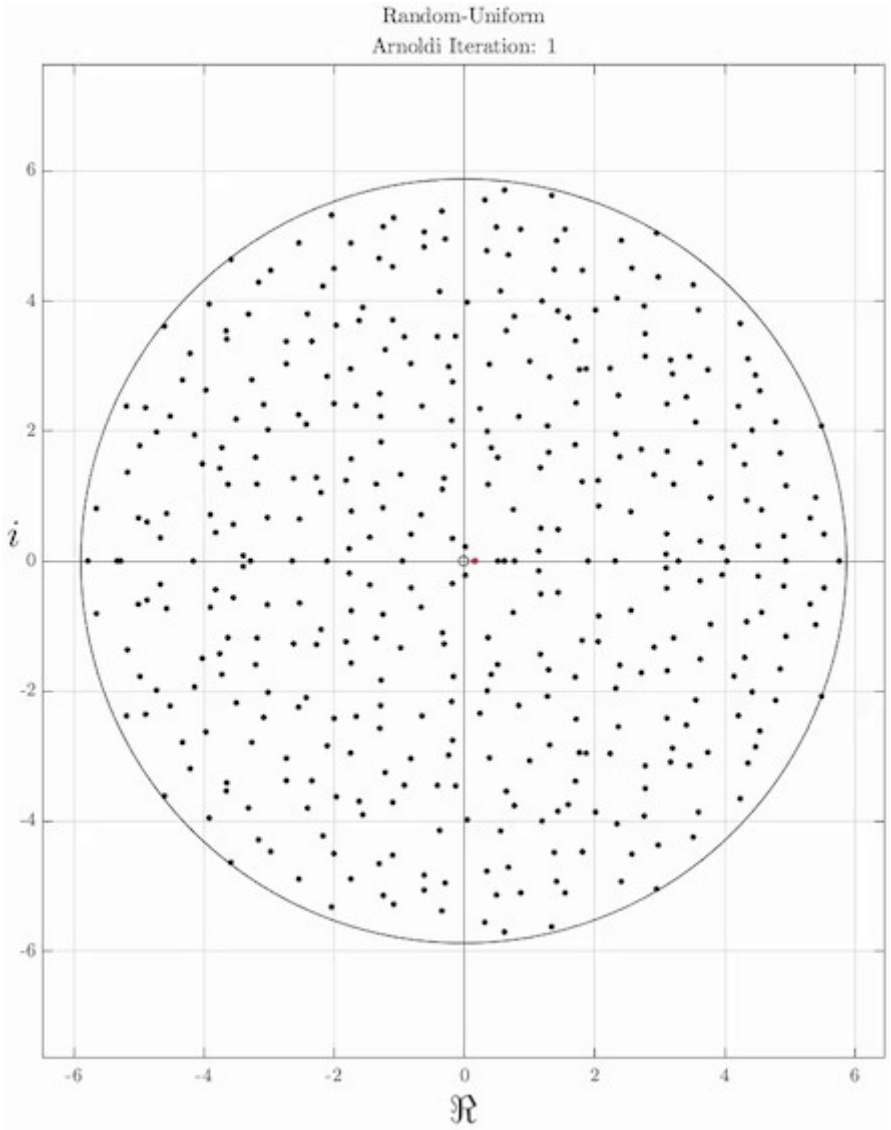
Finding eigenvalues with the Arnoldi iteration

The idea of the Arnoldi iteration as an eigenvalue algorithm is to compute the eigenvalues in the Krylov subspace. The eigenvalues of H_n are called the *Ritz eigenvalues*. Since H_n is a Hessenberg matrix of modest size, its eigenvalues can be computed efficiently, for instance with the QR algorithm, or somewhat related, Francis' algorithm. Also Francis' algorithm itself can be considered to be related to power iterations, operating on nested Krylov subspace. In fact, the most basic form of Francis' algorithm appears to be to choose b to be equal to Ae_1, and extending n to the full dimension of A. Improved versions include one or more shifts, and higher powers of A may be applied in a single steps.[2]

This is an example of the Rayleigh-Ritz method.

It is often observed in practice that some of the Ritz eigenvalues converge to eigenvalues of A. Since H_n is n-by-n, it has at most n eigenvalues, and not all eigenvalues of A can be approximated. Typically, the Ritz eigenvalues converge to the largest eigenvalues of A. To get the smallest eigenvalues of A, the inverse (operation) of A should be used instead. This can be related to the characterization of H_n as the matrix whose characteristic polynomial minimizes $\|p(A)q_1\|$ in the following way. A good way to get $p(A)$ small is to choose the polynomial p such that $p(x)$ is small whenever x is an eigenvalue of A. Hence, the zeros of p (and thus the Ritz eigenvalues) will be close to the eigenvalues of A.

However, the details are not fully understood yet. This is in contrast to the case where A is Hermitian. In that situation, the Arnoldi iteration becomes the Lanczos iteration, for which the theory is more complete.



Arnoldi iteration demonstrating convergence of Ritz values (red) to the eigenvalues (black) of a 400x400 matrix, composed of uniform random values on the domain [-0.5 +0.5]

Restarted Arnoldi iteration

Due to practical storage consideration, common implementations of Arnoldi methods typically restart after a fixed number of iterations. One approach is the Implicitly Restarted Arnoldi Method (IRAM)[3] by Lehoucq and Sorensen, which was popularized in the free and open source software package ARPACK.[4] Another approach is the Krylov-Schur Algorithm by G. W. Stewart, which is more stable and simpler to implement than IRAM.[5]

See also

The generalized minimal residual method (GMRES) is a method for solving $Ax = b$ based on Arnoldi iteration.

References

1. Arnoldi, W. E. (1951). "The principle of minimized iterations in the solution of the matrix eigenvalue problem" (https://www.ams.org/qam/1951-09-01/S0033-569X-1951-42792-9/) . *Quarterly of Applied Mathematics*. **9** (1): 17–29. doi:10.1090/qam/42792 (https://doi.org/10.1090%2Fqam%2F42792) . ISSN 0033-569X (https://search.worldcat.org/issn/0033-569X) .
2. David S. Watkins. Francis' Algorithm (http://math.wsu.edu/faculty/watkins/slides/ilas10.pdf) Washington State University. Retrieved 14 December 2022
3. R. B. Lehoucq & D. C. Sorensen (1996). "Deflation Techniques for an Implicitly Restarted Arnoldi Iteration". *SIAM Journal on Matrix Analysis and Applications*. **17** (4): 789–821. doi:10.1137/S0895479895281484 (https://doi.org/10.1137%2FS0895479895281484) . hdl:1911/101832 (https://hdl.handle.net/1911%2F101832) .
4. R. B. Lehoucq; D. C. Sorensen & C. Yang (1998). "ARPACK Users Guide: Solution of Large-Scale Eigenvalue Problems with Implicitly Restarted Arnoldi Methods" (https://web.archive.org/web/20070626080708/http://www.ec-securehost.com/SIAM/SE06.html) . SIAM. Archived from the original (http://www.ec-securehost.com/SIAM/SE06.html) on 2007-06-26. Retrieved 2007-06-30.
5. Stewart, G. W. (2002). "A Krylov--Schur Algorithm for Large Eigenproblems" (http://epubs.siam.org/doi/10.1137/S0895479800371529) . *SIAM Journal on Matrix Analysis and Applications*. **23** (3): 601–614. doi:10.1137/S0895479800371529 (https://doi.org/10.1137%2FS0895479800371529) . ISSN 0895-4798 (https://search.worldcat.org/issn/0895-4798) .

- W. E. Arnoldi, "The principle of minimized iterations in the solution of the matrix eigenvalue problem," *Quarterly of Applied Mathematics*, volume 9, pages 17–29, 1951.
- Yousef Saad, *Numerical Methods for Large Eigenvalue Problems*, Manchester University Press, 1992. ISBN 0-7190-3386-1.
- Lloyd N. Trefethen and David Bau, III, *Numerical Linear Algebra*, Society for Industrial and Applied Mathematics, 1997. ISBN 0-89871-361-7.
- Jaschke, Leonhard: *Preconditioned Arnoldi Methods for Systems of Nonlinear Equations*. (2004). ISBN 2-84976-001-3
- Implementation: Matlab comes with ARPACK built-in. Both stored and implicit matrices can be analyzed through the eigs() (http://www.mathworks.com/help/techdoc/ref/eigs.html) function.